

Lecture: 0101

Latches, Flip-Flops & Timing

From Combinational to Sequential

How a circuit gains memory — and how a clock keeps that memory sane.

CSA222: Modern Computer Architecture

Everything So Far Had No Memory

L1–L4 built powerful circuits — but they forget everything instantly.



Gates & minimisation

AND/OR/NOT, K-maps, universal NAND (L1–L2).



MUX, decoder, adder

Reusable blocks that build a datapath (L3).



Number representation

Two's complement, arithmetic, overflow (L4).



All 'stateless'

Output = $f(\text{inputs NOW})$.
Change inputs → output forgets the past.

A CPU needs to REMEMBER — the program counter, registers, state. Today: how circuits learn to hold a value.

Today's Roadmap

Feedback creates memory; a clock makes memory trustworthy.

1

Feedback = Memory

Loop a circuit's output back to its input, and it can hold a bit.

2

Latches

SR and D latches — the first storage elements, and their dangers.

3

Flip-Flops

Edge-triggered storage — the reliable building block of every register.

4

Timing

Setup, hold, and metastability — the rules that keep it all working.

The big shift: from 'output depends on inputs now' to 'output depends on inputs AND history'. That's the birth of state.

Combinational vs Sequential

The single most important distinction in digital design.

COMBINATIONAL

Output = $f(\text{inputs now})$

- No memory — purely reactive.
- Same inputs always give same output.
- Examples: adders, MUXes, decoders.
- The instant inputs change, output follows.

SEQUENTIAL

Output = $f(\text{inputs, past state})$

- Has memory — remembers history.
- Same inputs can give different outputs.
- Examples: registers, counters, FSMs.
- A clock decides WHEN state updates.

The difference is one thing: a feedback loop. Add it, and a circuit can remember.

1. Feedback = Memory

Route a circuit's output back into its own input, and something remarkable happens: it remembers.

The Idea of Feedback

Without a loop, signals flow one way and vanish. With a loop, they persist.

NO FEEDBACK (combinational)



Signal flows left → right, once. Remove the input and the output is gone.

WITH FEEDBACK (sequential)



↻ output feeds back

THE INSIGHT

A feedback loop lets the output 'hold itself up'.

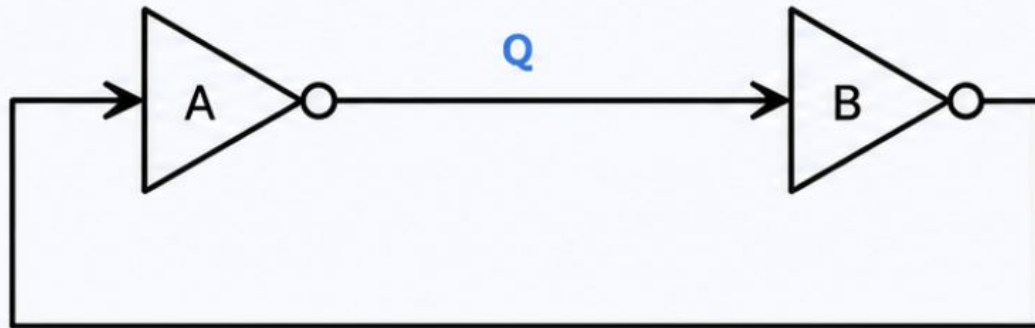
The circuit's current output becomes part of its own next input — so it can sustain a value even after the original input is gone.

That self-sustaining value IS a stored bit. Memory, from nothing but a loop.

The Bistable Element

Two inverters in a loop = the simplest 1-bit memory. It has two stable states.

Feedback Loop Diagram



Output wire feeds back to input — circuit holds its last stable value

If $Q = 1$, the loop keeps $Q = 1$. If $Q = 0$, it keeps $Q = 0$. Both are stable — hence 'bi-stable'.

TWO STABLE STATES

This holds a bit forever (while powered) — but there's a catch:

there's no way to **CHANGE** it. It's sealed.

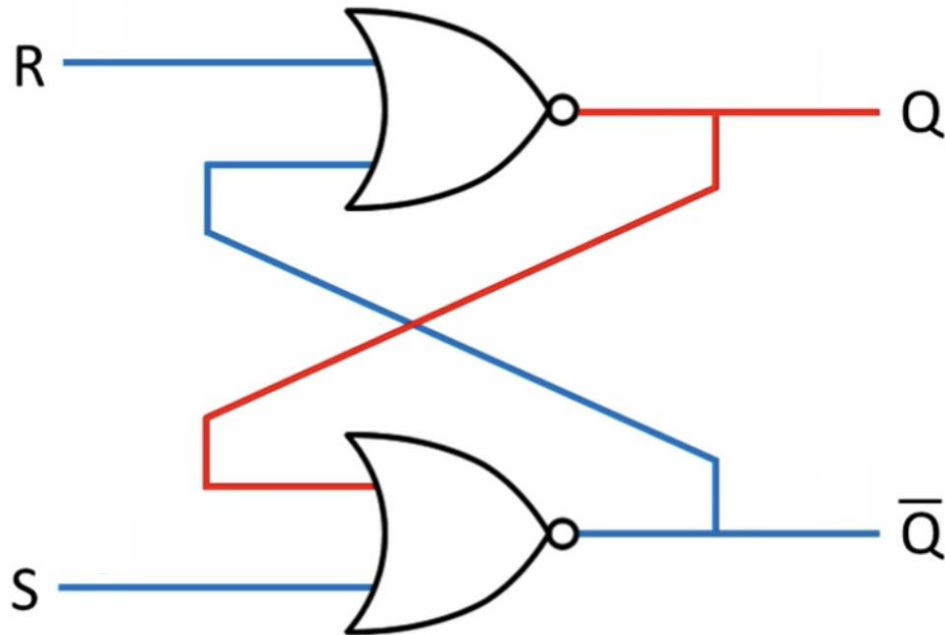
To make it useful, we need controlled inputs to set and reset the stored value. That's exactly what a latch adds.

2. Latches

Add controllable inputs to the feedback loop — now you can set, reset, and hold a bit on demand.

The SR Latch (NOR-based)

Two cross-coupled NOR gates: Set makes $Q=1$, Reset makes $Q=0$.



Each gate's output feeds the OTHER gate's input — that's the cross-coupling that holds state.

HOW IT BEHAVES

$S = 1 \rightarrow$ Set: Q becomes 1

$R = 1 \rightarrow$ Reset: Q becomes 0

$S = R = 0 \rightarrow$ Hold: Q keeps its value

The 'hold' state is the memory — the latch remembers whatever it was last set to.

SR Latch: The Four States

Every combination of S and R, and what the latch does.

S	R	Action	Result
0	0	Hold	Q unchanged — remembers last value
1	0	Set	Q = 1
0	1	Reset	Q = 0
1	1	Forbidden	Q and Q' both 0 — invalid!

Three useful states — Set, Reset, Hold — plus one you must never allow. That forbidden state is next.

The Forbidden State

$S = R = 1$ breaks the latch's fundamental promise: Q and Q' are supposed to be opposites.

WHAT GOES WRONG

With $S = R = 1$, BOTH NOR gates output 0.

So $Q = 0$ AND $Q' = 0$ — but Q' is supposed to be NOT Q . The invariant is broken.

Worse: if S and R drop to 0 at the same instant, the latch enters a RACE — its final state is unpredictable, decided by tiny gate-delay differences.

WHY YOU CARE

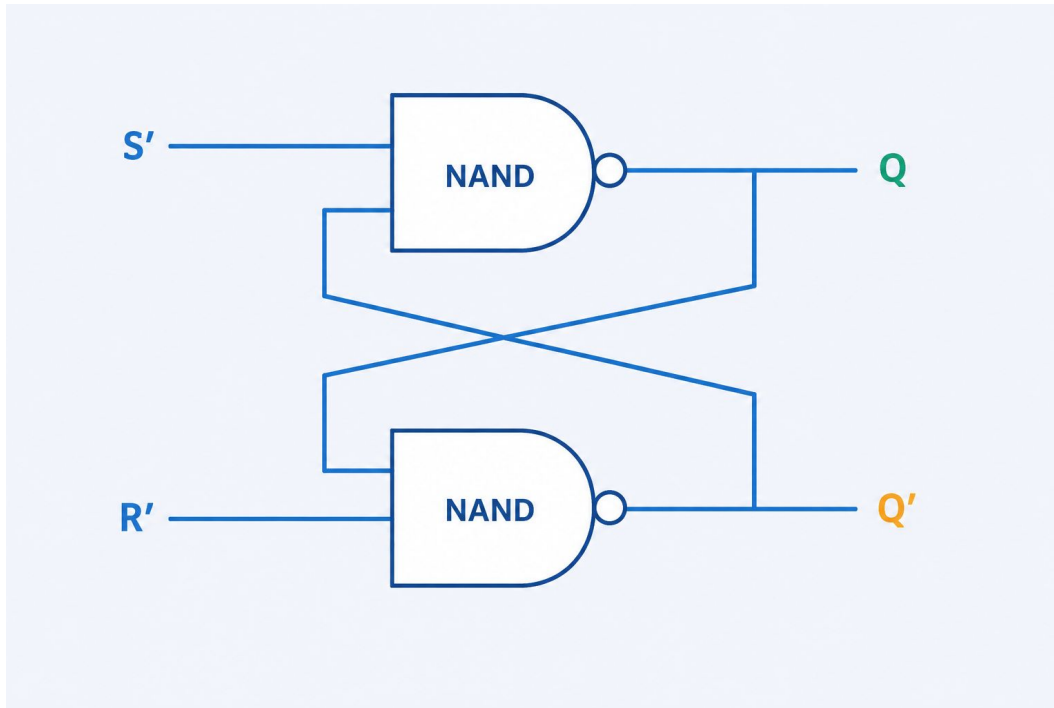
This is a real design hazard.

It's why raw SR latches are rarely used directly — designers add logic to make $S = R = 1$ impossible.

The D latch (two slides away) solves this elegantly: it makes S and R always opposite, so the forbidden state simply can't occur.

SR Latch, NAND Version

Same idea, built from NAND gates — but the inputs are active-LOW.



Note the primes: S' and R' are active-LOW — a 0 triggers the action, not a 1.

NOR vs NAND LATCH

NOR latch: active-HIGH (1 = act).

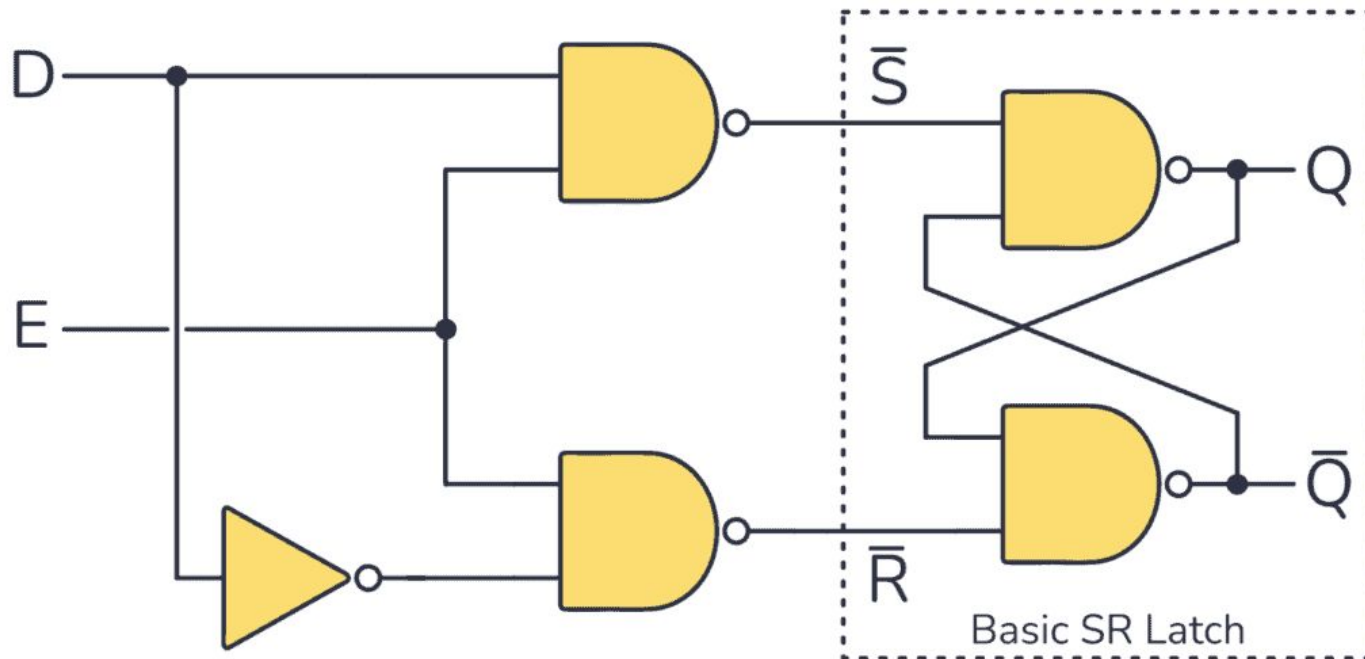
NAND latch: active-LOW (0 = act).

Forbidden state flips too: for the NAND latch, $S' = R' = 0$ is the illegal one.

NAND latches are common in practice because NAND is the cheap, universal gate (recall L2).

The D Latch

One data input, no forbidden state. The forbidden combination is engineered away.



BEHAVIOUR

$EN = 1 \rightarrow Q$ follows D .
(transparent — $Q = D$)

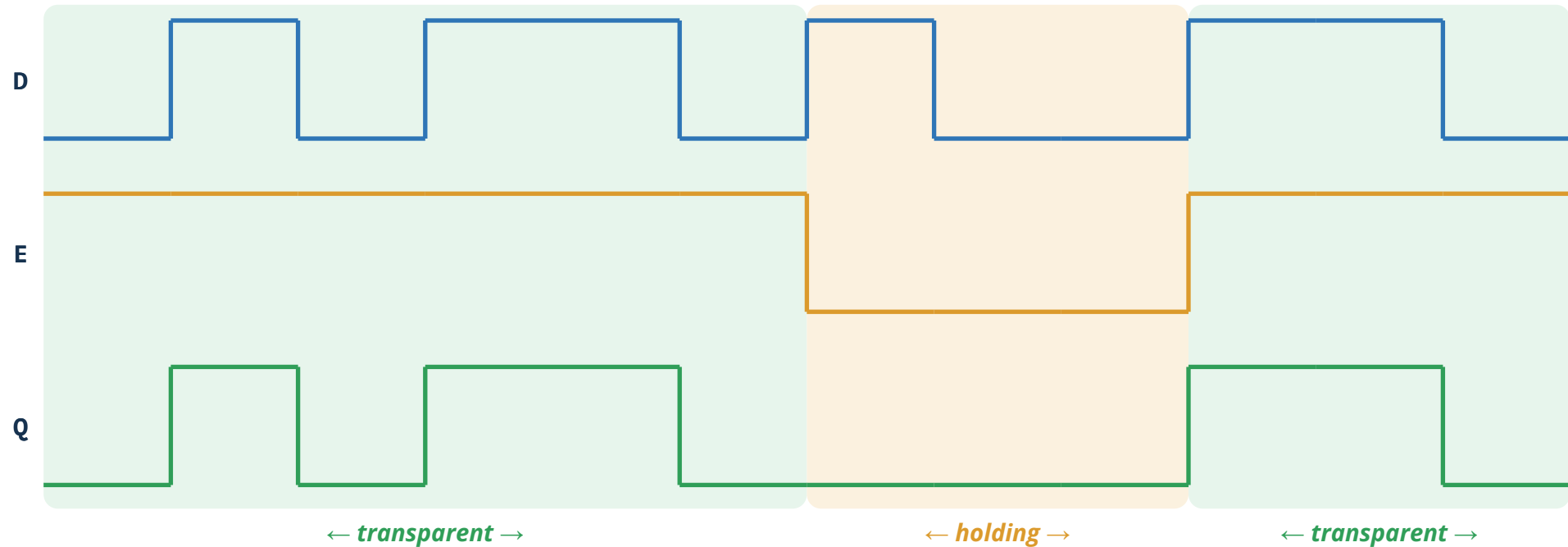
$EN = 0 \rightarrow Q$ holds its last value.
(opaque — ignores D)

One clean data input, one enable, zero forbidden states. This is genuinely useful memory.

*D drives S directly, and $\text{NOT-}D$ drives R — so S and R are ALWAYS opposite.
Forbidden state: impossible.*

D Latch · Transparency

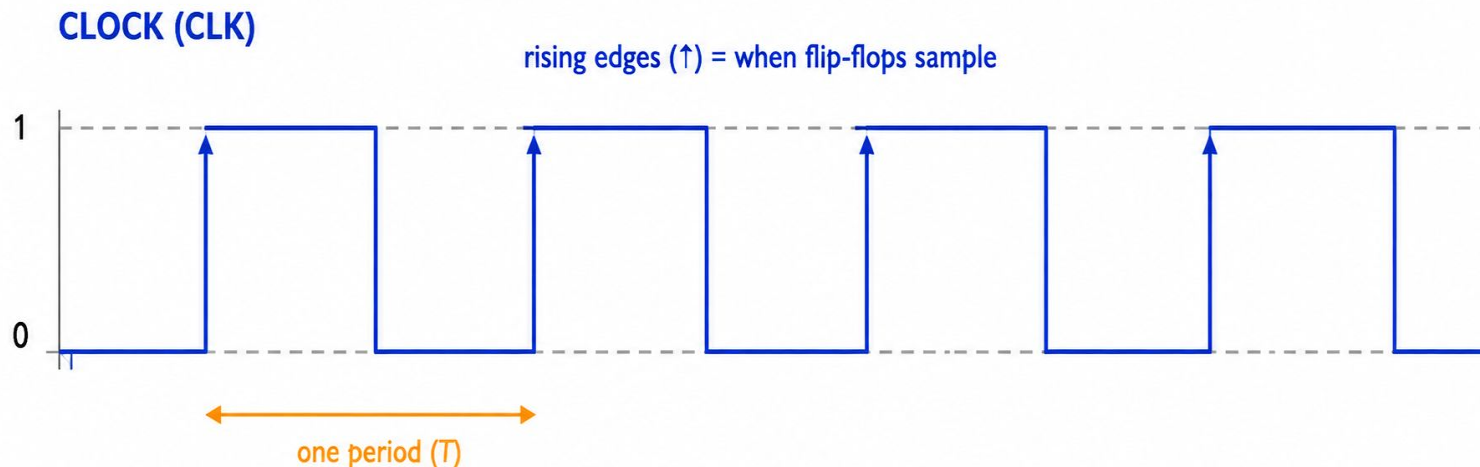
While E is high, Q tracks D step-for-step. While E is low, whatever D was at the falling edge of E is what Q holds.



During the green windows, Q is a copy of D. During the amber window, Q remembers the last D from before E fell.

The Clock: The System Heartbeat

One periodic signal synchronises every flip-flop in the chip.



Frequency: $f = 1 / T$

Example: A 3 GHz CPU has $T = 0.33$ nanoseconds $\rightarrow$ 3 billion edges per second.

WHY ONE CLOCK

Every flip-flop updates on the same edge.

That shared instant is what keeps a whole chip's state consistent — everyone steps together.

The clock is the drummer the entire orchestra follows. Lose sync, lose everything.